

ThermogramBaseline Demo

In order to do some of the work below, we must have the tidyverse package installed.

```
library(tidyverse, quietly = TRUE)
#> -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
#> v dplyr      1.1.4      v purrr      1.0.2
#> v forcats    1.0.0      v readr      2.1.5
#> v ggplot2    4.0.1      v tibble     3.2.1
#> v lubridate  1.9.3      v tidyr      1.3.1
#> -- Conflicts ----- tidyverse_conflicts() --
#> x dplyr::filter() masks stats::filter()
#> x dplyr::lag()     masks stats::lag()
#> i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Here we load the package and look at the data. Data must be formatted in this way:

```
library(ThermogramBaseline)
```

```
data(DemoBaseline, package = "ThermogramBaseline")
head(DemoBaseline)
```

SampleID	Temperature	dCp
3a	20.00018	0.0899742
3a	20.03254	-0.0816733
3a	20.08539	-0.0622706
3a	20.15371	0.0106867
3a	20.25106	0.1488339
3a	20.34125	0.1822992

Each function can only take in one thermogram sample at a time so we must filter for sample 11b and filter the temperature to between 45 and 90

```
SampleIDs <- DemoBaseline %>% pull(SampleID) %>% unique() %>% as.vector()

Sample.1 <- DemoBaseline %>% filter(SampleID == SampleIDs[2]) %>%
  filter(between(Temperature, 45,90))
```

Now that we have our first sample, lets get our two endpoints for baseline detection.

```
endpoints <- endpoint.detection(x = Sample.1, exclusion.lwr = 60,
                                exclusion.upr = 80, point.selection = "innermost",
                                explicit = FALSE)
```

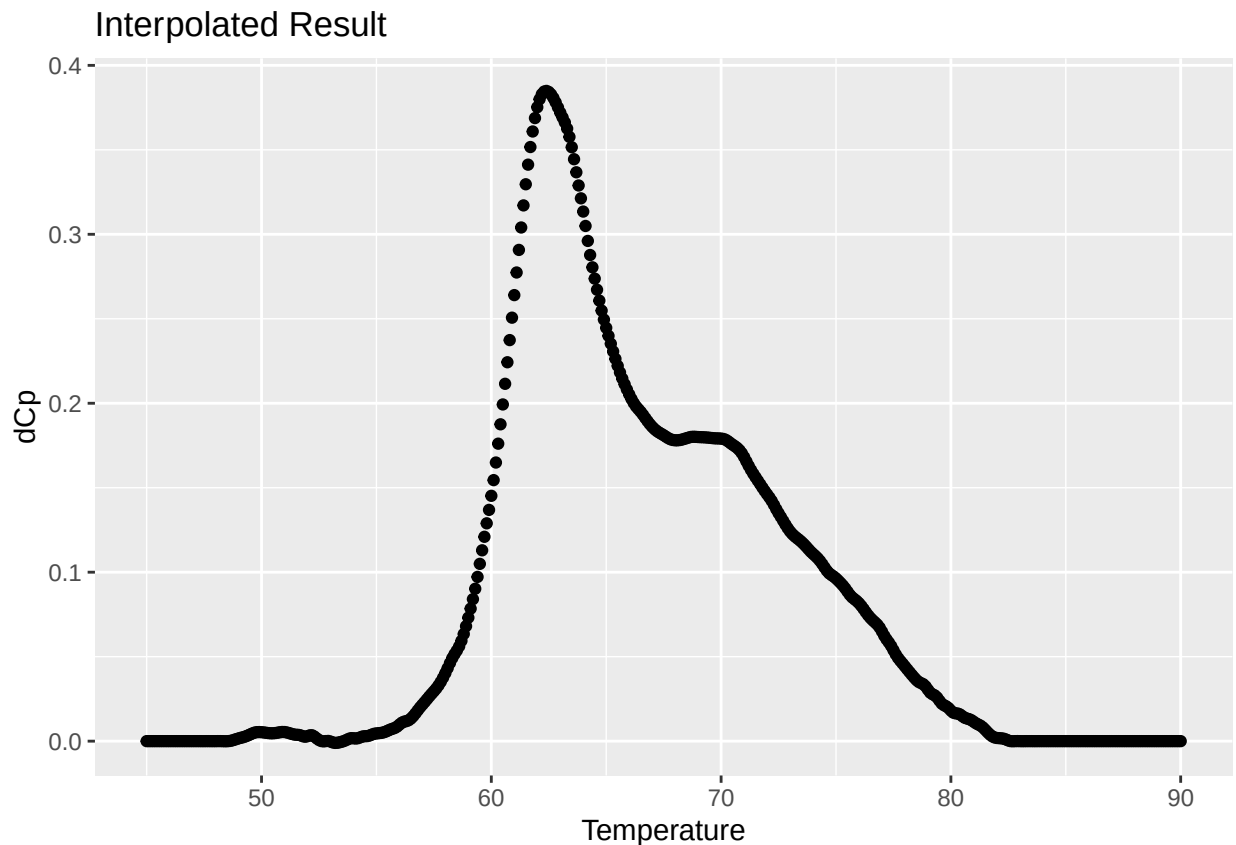
lower	upper	method
48.63918	82.55743	innermost

The model selected the temperatures 48.63918 and 82.55743 as our endpoints for our baseline subtraction. Next lets subtract the baseline using these endpoints.

```
baseline <- baseline.subtraction.byhand(x= Sample.1, lwr.temp = endpoints$lower,
                                       upr.temp = endpoints$upper, plot.on = FALSE)
```

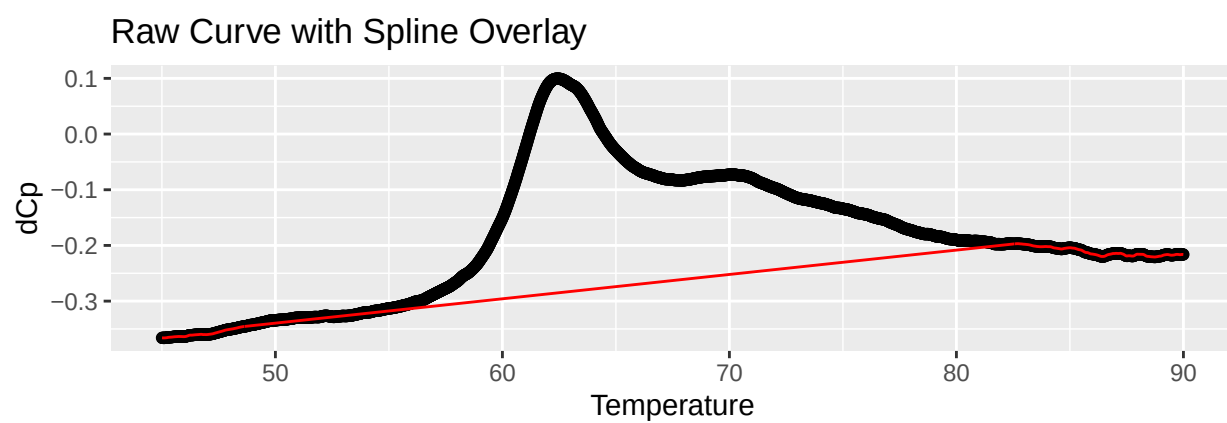
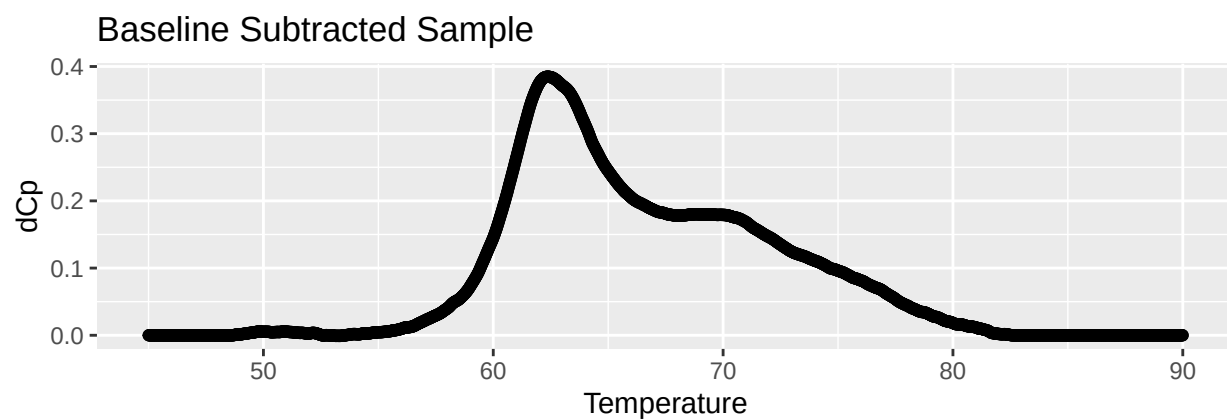
Now that we have subtracted the baseline, we will interpolate the data onto a grid of set temperatures. Notice plot.on is true by default.

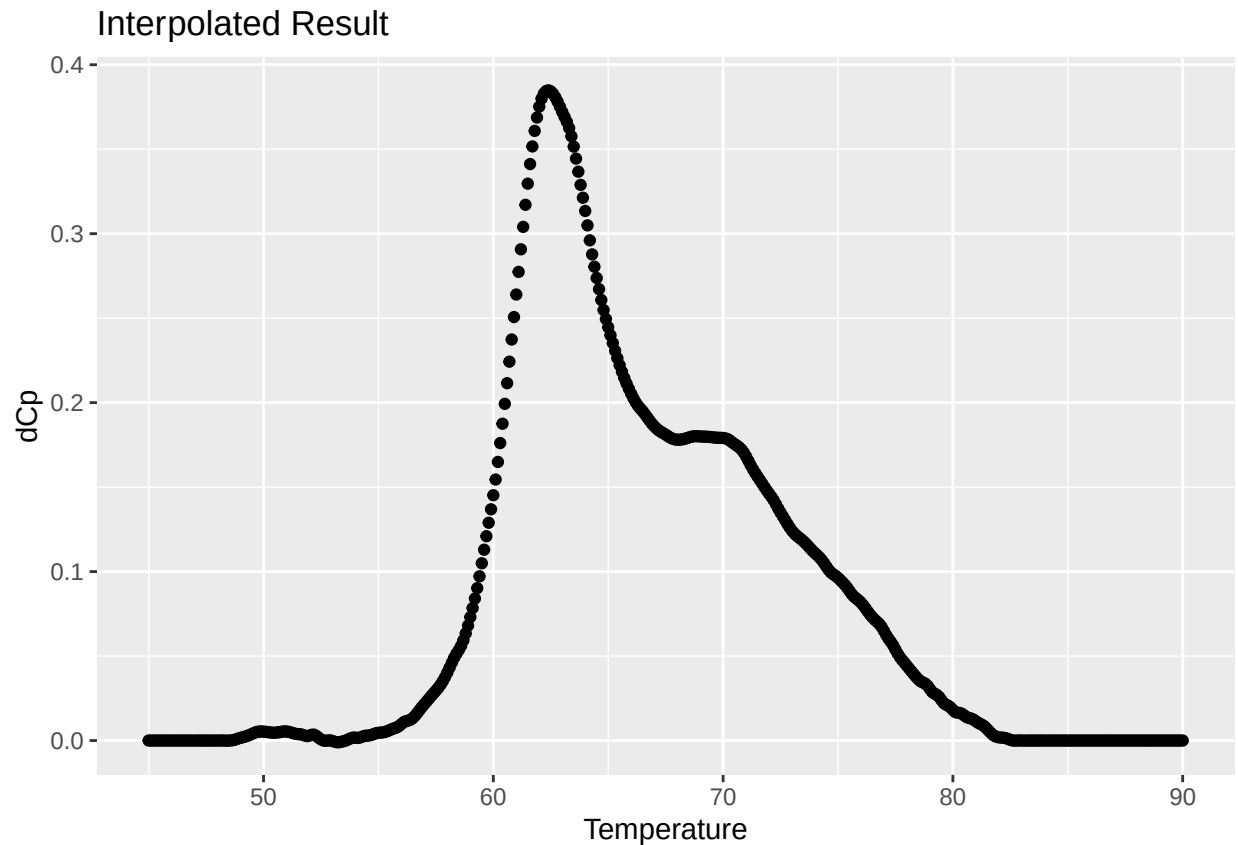
```
final <- final.sample.interpolate(x=baseline,grid.temp = seq(from = 45,to = 90,by = 0.1))
```



Here we have our final baseline subtracted interpolated sample. We can also use the auto function to complete these in one step. Notice that the plot.on default is false so we must set it to true if we want graphs to appear.

```
auto <- auto.baseline(x=Sample.1, exclusion.lwr = 60, exclusion.upr = 80,
                     grid.temp = seq(45,90,0.1),plot.on = TRUE, explicit = FALSE)
```





Lastly, lets run all 6 thermograms through our function.

```
### Set our grid of temperatures to interpolate onto
grid.temp <- seq(from = 45,to = 90, by = 0.1)

### Create empty data frame to store results
Final.Results <- data.frame(Temperature = grid.temp)

### Store all sample IDs and how many samples we need to analyze.
All.IDs <- DemoBaseline %>% pull(SampleID) %>% unique() %>% as.vector()
n.samples <- length(All.IDs)

### For loop for running all the samples through the algorithm
for(j in 1:n.samples)
{
  cat('Working on Sample ', All.IDs[j], 'element ', j, ' of ', n.samples, ' \n')
  ### select a sample
  working.sample <- DemoBaseline %>%
    filter(SampleID == All.IDs[j]) %>%
    filter(between(Temperature, 45,90))
  ### get a baseline-subtracted and interpolated final result!
  auto.output <- auto.baseline(x = working.sample, grid.temp = grid.temp, plot.on = FALSE)
  Final.Results <- Final.Results %>% cbind(out = auto.output$dCp)
  cat("\014")
}
```

```

### Rename the column names to the correct
colnames(Final.Results)[-1] <- All.IDs

### The below line will save the resulting data frame as a csv file
### Only run this line once
#write.csv(x = Final.Results, file = 'Final.Results.csv')

```

The code above provides a data frame with 6 baseline subtracted thermograms interpolated all to the same temperature grid. Below it is shown how to produce a graph of each thermogram output.

```

pdf('/Final.Results.pdf')
{
  for(j in 1:length(All.IDs))
  {
    g1 <- Final.Results %>% select(Temperature, All.IDs[j]) %>%
      ggplot(aes(x = Temperature, y = .[,2])) +
      geom_point() +
      labs(title = paste0('Final Automated Sample for ', str_sub(All.IDs[j], 2)))
    print(g1)
  }
}
dev.off()

```